

PROJET DE PROGRAMMATION ORIENTÉE OBJET EN PHP

« Kaay Dem ! » — Plateforme de covoiturage étudiant

Durée : 2 semaines • Groupes de 3 étudiants • PHP natif (sans framework)

1. Contexte

Chaque jour, des centaines d'étudiants effectuent le trajet entre Dakar, Rufisque, Diamniadio et les campus environnants, souvent avec des difficultés de transport et des coûts élevés. Votre groupe est chargé de développer « Kaay Dem ! », une plateforme web de covoiturage réservée à la communauté étudiante : les conducteurs publient leurs trajets, les passagers réservent des places, et chacun évalue l'autre après le voyage.

L'application sera développée intégralement en PHP orienté objet, sans framework, afin de démontrer votre maîtrise des concepts fondamentaux de la POO et d'une architecture MVC construite à la main.

2. Objectifs pédagogiques

À l'issue de ce projet, vous devez être capable de :

- Modéliser un problème réel sous forme de classes et d'objets (diagramme de classes UML) ;
- Mettre en œuvre l'encapsulation, l'héritage, le polymorphisme, les classes abstraites et les interfaces ;
- Structurer une application selon le patron MVC avec des espaces de noms (namespaces)
- Manipuler une base de données via PDO et le patron Repository ;
- Gérer les erreurs avec des exceptions personnalisées ;
- Collaborer en équipe avec Git/GitHub (branches, pull requests, commits conventionnels).

3. Description fonctionnelle

3.1 Acteurs

Acteur	Rôle
Conducteur	Publie des trajets (départ, arrivée, date/heure, places, tarif), accepte ou refuse les réservations, clôture le trajet.
Passager	Recherche des trajets, réserve des places, annule, évalue le conducteur après le voyage.
Administrateur	Valide les comptes conducteurs (permis, véhicule), gère les utilisateurs et les signalements d'abus, consulte les statistiques.

Un même utilisateur peut être à la fois conducteur et passager : votre modélisation objet devra gérer ce double rôle proprement (héritage, interfaces ou composition — justifiez votre choix dans le rapport).

3.2 Fonctionnalités obligatoires

- **Authentification** : inscription, connexion/déconnexion par sessions, mots de passe hachés (password_hash), contrôle d'accès par rôle ; un conducteur doit être validé par l'administrateur avant de publier.
- **Gestion des trajets** : publication (départ, arrivée, points d'arrêt, date/heure, nombre de places, prix par place), modification et annulation tant qu'aucune réservation n'est confirmée.
- **Recherche** : recherche de trajets par ville de départ, ville d'arrivée et date, avec filtres (prix maximum, places disponibles) et pagination.
- **Réservations** : cycle de vie En attente → Confirmée → Terminée ou Annulée ; décrémentation automatique des places ; un passager ne peut pas réserver deux trajets qui se chevauchent ; chaque transition est historisée.
- **Évaluations** : après un trajet terminé, le passager note le conducteur (1 à 5 étoiles + commentaire) ; la note moyenne s'affiche sur le profil du conducteur.
- **Tableaux de bord** : conducteur (mes trajets, réservations reçues), passager (mes réservations), administrateur (statistiques : trajets par mois, taux d'occupation, top conducteurs).

3.3 Fonctionnalités bonus (facultatives)

- Messagerie interne conducteur-passager ; notification par e-mail à la confirmation ; calcul automatique du prix suggéré selon la distance ; export CSV ; petite API REST JSON (lecture seule) ; carte des trajets (Leaflet).

4. Contraintes techniques

Le non-respect de ces contraintes sera pénalisé dans la notation :

- **POO obligatoire** : au moins une classe abstraite, deux interfaces (ex. RepositoryInterface, EvaluableInterface), de l'héritage justifié, des propriétés typées, un trait (ex. Timestampable) et une énumération (enum) pour les statuts de réservation.
- **Architecture MVC** : séparation stricte Contrôleurs / Modèles (entités + repositories) / Vues ; un point d'entrée unique (index.php) avec un routeur maison.
- **Base de données** : MySQL ou MariaDB, accès exclusivement via PDO avec requêtes préparées ; script SQL de création fourni.
- **Exceptions personnalisées** : ex. PlacesInsuffisantesException, ReservationConflictException, gérées proprement (pages d'erreur).
- **Sécurité minimale** : échappement des sorties (htmlspecialchars), requêtes préparées, protection des routes selon le rôle.
- **Git/GitHub** : dépôt unique par groupe, commits réguliers de chaque membre, travail par branches et pull requests.

5. Livrables

1. Code source complet sur GitHub (lien à fournir), avec un README expliquant l'installation et les comptes de test ;
2. Script SQL de création et de peuplement de la base (données de démonstration) ;
3. Rapport technique (8 à 12 pages) : diagramme de classes UML, choix de conception (notamment la gestion du double rôle conducteur/passager), répartition des tâches, difficultés rencontrées ;
4. Démonstration fonctionnelle lors de la soutenance (10 min de présentation + 5 min de questions).

6. Planning indicatif sur 2 semaines

Période	Activités	Résultat attendu
Jours 1-2	Analyse, diagramme de classes UML, modèle de données, maquettes écrans, mise en place du dépôt Git	Conception validée par l'enseignant
Jours 3-5	Socle technique : autoloader PSR-4, routeur, connexion PDO, entités et repositories, authentification et validation des conducteurs	Connexion et inscription fonctionnelles
Jours 6-9	Publication et recherche de trajets, cycle de vie des réservations, gestion des places et des conflits	Cœur applicatif fonctionnel
Jours 10-12	Évaluations, tableaux de bord, statistiques, gestion des exceptions, sécurité	Application complète
Jours 13-14	Tests, données de démonstration, rapport, préparation de la soutenance	Livrables finaux

7. Barème de notation (sur 20)

Critère	Points	Évalué via
Qualité de la conception objet (UML, abstraction, interfaces, héritage, polymorphisme, gestion du double rôle)	5 pts	Rapport + code
Architecture MVC, routeur, autoloader PSR-4, organisation du code	4 pts	Code
Fonctionnalités obligatoires complètes et correctes	5 pts	Démonstration
Base de données, PDO, sécurité, gestion des exceptions	3 pts	Code + démo
Travail collaboratif Git (commits de chaque membre, branches, PR)	1 pt	Historique GitHub
Rapport et soutenance (clarté, maîtrise individuelle du code)	2 pts	Soutenance
Bonus (fonctionnalités facultatives)	+2 pts max	Démonstration

Attention : la note tient compte de la contribution individuelle, vérifiée par l'historique Git et par des questions personnalisées en soutenance. Tout plagiat ou code généré non maîtrisé entraînera une convocation devant le conseil pédagogique.

8. Conseils

- Commencez simple : faites fonctionner le cycle complet publication → réservation → confirmation → évaluation avant d'ajouter des options ;
- Validez votre diagramme de classes auprès de l'enseignant avant de coder (fin du jour 2) — la modélisation du double rôle est le point clé du projet ;
- Committez petit et souvent ; ne travaillez jamais à plusieurs sur le même fichier sans branche ;
- Répartissez les rôles (chef de projet, base de données, backend, vues) mais faites tourner les tâches pour que chacun touche à la POO.